Pramaan

Running the project without Docker (Windows / PowerShell)
AI-Run Statutory Revenue Audit · run guide with explanations

What you are starting

Pramaan is three cooperating parts. Running it by hand just means starting each one:

• **Backend API** (FastAPI, port 8000) – the audit engine, authorization and the hash-chained log.

• **Frontend** (React/Vite, port 5173) – the human auditor + client workspace; it proxies /api to the backend.

• **MCP server** (optional, port 8001) – the door an AI agent uses to drive the audit with an identity.

A clean, fully-loaded demo instance (synthetic company imported, sales created, one audit already run) is produced by a single reset command, so you are never starting from an empty database.

Prerequisites

Python 3.11 and Node.js installed, and the project at the path below. All commands are run from the **ELEV19** folder (the parent of **Pramaan**) unless noted – that is required for Python to import the **Pramaan** package.

One-time setup

Install backend and frontend dependencies (and, only if you will use the Groq agent, create a .env):

Step 1 — Load a clean demo instance

What it does: creates **Pramaan\.demo\runtime**, imports the synthetic company data, creates ~26 sales plus the hero case, runs one audit, and issues client/auditor/human tokens. It prints the run_id and seed.

Step 2 — Get your sign-in tokens

What it does: shows the client, auditor and human-reviewer tokens (valid 15 minutes). Copy the auditor token for the UI. These identities are the whole security model – an auditor can read everything and change nothing.

Step 3 — Start the backend (Terminal A)

The environment variable is the key glue: it points the server at the demo instance you just loaded.

What it does: runs the API on http://127.0.0.1:8000 with the authorization middleware and the audit engine behind it.

Step 4 — (Optional) a longer-lived token (Terminal B)

Demo tokens last 15 minutes; mint a 1-hour one against the same instance (TTL is capped at 3600s):

Step 5 — Start the auditor workspace (Terminal C)

What it does: serves the UI on http://127.0.0.1:5173 and proxies /api to the backend on 8000. Open it, paste the auditor token, and you are in. This is the human surface; the runs, findings, hero sale, event log and exports are all here.

Step 6 — (Optional) the MCP server for an AI agent (Terminal D)

What it does: exposes the audit tools over MCP so an AI agent can drive the system with an identity. Not needed for the stdio agent console below.

Step 7 — (Optional) drive it with the Groq agent console (Terminal E)

Needs .env with GROQ_API_KEY and PRAMAAN_TOKEN set (the auditor token). Run from ELEV19:

What it does: lists the tools the identity is allowed, runs the audit, and returns an explanation – with sensitive fields redacted and only evidence-cited sentences kept.

Step 8 — Metrics and tests (no running server needed)

What it does: the first prints the measured detection numbers (8/8 vs baseline 3/8, 0 false alarms); the second runs the core test suites.

How it works, in one paragraph

The client uploads a sales register, related parties and evidence. The auditor creates a run, which snapshots and hashes the data for reproducibility, then risk-aware sampling selects sales (each with a written reason) and the deterministic tests T00–T15 produce findings with states and evidence references. Evidence is tagged by who controls it (the witness model). A human – never the AI – records the final outcome, and the workpaper, memo and defensibility pack are exported with a receipt so the result can be recomputed by hand. The same backend is reachable two ways – the web UI for humans and MCP for an AI agent – and both enforce identical identity rules.

Gotchas

• Run Python commands from **ELEV19** (the parent of Pramaan), or imports fail.

• Set **$env:PRAMAAN_DATA** in every terminal that talks to the demo data (Steps 3, 4, 6).

• Tokens expire in 15 minutes – re-run reset or issue a fresh one.

• “port already allocated / in use” means something else is on 8000 – stop it or use another port.

• To return to a clean state between rehearsals, re-run **python -m Pramaan.backend.demo reset**.

Quick reference

Goal	Command (from ELEV19)
Load clean demo	`python -m Pramaan.backend.demo reset`
Backend API	`uvicorn Pramaan.backend.app:app --port 8000`
Frontend UI	`cd Pramaan\web ; npm run dev`
MCP server	`python -m Pramaan.backend.mcp_server --http`
Groq agent	`python -m Pramaan.backend.agent_llm "..."`
Metrics	`python -m Pramaan.backend.evaluate`

Pramaan – synthetic/local prototype. See DEMO_SCRIPT.md, CLIENT_DEMO.md and PROJECT_REPORT.md for more.